

Operating Systems - Chapter 29 Summary

Lock-Based Concurrent Data Structures

Concurrent data structures require careful use of locks to maintain correctness while maximizing performance. This chapter illustrates how locking strategies affect scalability and responsiveness in shared counters, lists, queues, and hash tables.

1. Shared Counter

```
typedef struct __counter_t {
    int value;
    pthread_mutex_t lock;
} counter_t;

void increment(counter_t *c) {
    pthread_mutex_lock(&c->lock);
    c->value++;
    pthread_mutex_unlock(&c->lock);
}
```

Simple and correct, but every increment serializes on a single lock. Scales poorly with many threads.

2. Approximate Counter

```
typedef struct __counter_t {
    int global;
    pthread_mutex_t glock;
    int local[NUMCPUS];
    pthread_mutex_t llock[NUMCPUS];
    int threshold;
} counter_t;

void update(counter_t *c, int tid, int amt) {
    int cpu = tid % NUMCPUS;
    pthread_mutex_lock(&c->llock[cpu]);
    c->local[cpu] += amt;
    if (c->local[cpu] >= c->threshold) {
        pthread_mutex_lock(&c->glock);
        c->global += c->local[cpu];
        pthread_mutex_unlock(&c->glock);
        c->local[cpu] = 0;
    }
    pthread_mutex_unlock(&c->llock[cpu]);
}
```

Improves scalability by using per-CPU local counters that flush to the global count periodically. The tradeoff is temporary inaccuracy in the counter value.

3. Concurrent Linked List

```
typedef struct __node_t {
    int key;
    struct __node_t *next;
    pthread_mutex_t lock;
} node_t;

int List_Insert(node_t **head, int key) {
    node_t *new = malloc(sizeof(node_t));
    new->key = key;
    pthread_mutex_lock(&(*head)->lock);
    new->next = *head;
```

```

        *head = new;
        pthread_mutex_unlock(&(*head)->lock);
        return 0;
    }

```

Coarse-grained locking (one lock for the whole list) ensures safety but forces serialization. Fine-grained locking, where each node has its own lock, allows concurrent operations on disjoint list regions.

4. Two-Lock Queue (Michael & Scott)

```

typedef struct __node_t {
    int value;
    struct __node_t *next;
} node_t;

typedef struct __queue_t {
    node_t *head;
    node_t *tail;
    pthread_mutex_t headLock, tailLock;
} queue_t;

void Enqueue(queue_t *q, int value) {
    node_t *tmp = malloc(sizeof(node_t));
    tmp->value = value;
    tmp->next = NULL;
    pthread_mutex_lock(&q->tailLock);
    q->tail->next = tmp;
    q->tail = tmp;
    pthread_mutex_unlock(&q->tailLock);
}

int Dequeue(queue_t *q, int *value) {
    pthread_mutex_lock(&q->headLock);
    node_t *tmp = q->head;
    node_t *newHead = tmp->next;
    if (newHead == NULL) {
        pthread_mutex_unlock(&q->headLock);
        return -1; // empty
    }
    *value = newHead->value;
    q->head = newHead;
    pthread_mutex_unlock(&q->headLock);
    free(tmp);
    return 0;
}

```

This queue allows concurrent enqueues and dequeues by protecting head and tail separately. It improves concurrency while maintaining simplicity.

5. Concurrent Hash Table

```

typedef struct __hash_t {
    list_t buckets[BUCKETS];
    pthread_mutex_t locks[BUCKETS];
} hash_t;

void Hash_Insert(hash_t *H, int key) {
    int bucket = key % BUCKETS;
    pthread_mutex_lock(&H->locks[bucket]);
    List_Insert(&H->buckets[bucket], key);
    pthread_mutex_unlock(&H->locks[bucket]);
}

```

Locking per bucket enables high concurrency when threads access different buckets. However, locking granularity increases memory use and code complexity.

Performance Lessons

- Coarse-grained locks: simple, but only one thread makes progress.
- Fine-grained locks: more concurrency, but complex and error-prone.
- Sharding and batching: practical balance between simplicity and scalability.
- More threads $\neq$ more performance: lock contention and cache coherence overheads dominate.

Summary

Lock-based data structures illustrate the trade-offs in concurrent design. Fine-grained and partitioned locks enable parallelism but complicate correctness. Understanding contention, deadlocks, and scalability limits is key to writing efficient concurrent systems.